

CONFIDENCE_MODEL.ipynb – Spiegazione dettagliata

Generato: 2026-05-08 11:59

Notebook: c:\Users\david\Desktop\Polito\Tesi_Davide\CONFIDENCE_MODEL.ipynb

Obiettivo del notebook

Il notebook carica un modello Keras (ConvNeXt) e un dataset (H5), esegue predizioni sul set di test, calcola confidence/errore ad alta confidenza, esporta un CSV con le probabilità per immagine e svolge analisi/plot downstream (uncertainty, agreement/IOA).

Input principali (path)

- Modello ConvNeXt: **/content/drive/MyDrive/Colab Notebooks/HPC/finale/model/pretrained_ConvNeXt_finetuning**
- Dataset train/val (cartella): **/content/drive/MyDrive/Colab Notebooks/HPC/finale/dataset** (usa dataset.h5)
- Dataset test (H5): **/content/drive/MyDrive/Colab Notebooks/datasets/dataset_giusto/test_path.h5**

Flusso logico (in breve)

- Setup ambiente (Colab/Drive + install dipendenze) se presente.
- Definizione di layer custom e loss (focal loss) per compatibilità nel load del modello.
- Caricamento ConvNeXt (tf.keras.models.load_model) con custom_object_scope.
- Caricamento dataset da file H5: X_train/X_val/X_test + label e (opzionalmente) paths delle immagini nel test.
- Creazione data generator bilanciato (con label smoothing) e generatore test con paths_data.
- Valutazione: predict → y_pred/y_true → errori ad alta confidenza + salvataggio grid immagini.
- Salvataggio risultati in CSV: per ogni immagine salva prob. per classe.
- Analisi downstream da CSV (uncertainty, agreement/IOA, plot).

Dettaglio per cella

Cella 1 – Setup Colab / Drive

Monta Google Drive e cambia directory per accedere ai file su /content/drive. In esecuzione locale queste righe non sono utilizzabili.

Estratto (prime righe non vuote):

```
from pathlib import Path
import sys
# Config path (Colab vs VS Code locale)
IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    ROOT = Path('/content/drive/MyDrive/Colab Notebooks/HPC/finale')
else:
    # In locale: metti qui la cartella base dove hai modello/dataset (default = cartella del work
```

Cella 2 – Installazione dipendenze

Installa TensorFlow (tipicamente solo su Colab). In locale va fatto nel tuo ambiente/venv una volta sola.

Estratto (prime righe non vuote):

```
# In locale evita di reinstallare TensorFlow ad ogni run.
# Se ti serve forzare TF 2.15, installa UNA volta nel tuo venv: `pip install tensorflow==2.15.0`
try:
```

```

import tensorflow as tf
print('TensorFlow:', tf.__version__)
except Exception as e:
    raise RuntimeError(
        'TensorFlow non trovato in questo kernel. Installa nel tuo ambiente e riavvia il kernel.'
    ) from e
# Info utili per capire la lentezza (CPU vs GPU)

```

Cella 3 – Markdown

CARICA I MODELLI

Cella 4 – Loss (Focal) + caricamento modello

Definisce la focal loss e carica il modello ConvNeXt da convnext_path usando custom_object_scope (necessario perché il modello usa oggetti custom).

Estratto (prime righe non vuote):

```

import tensorflow as tf
from tensorflow.keras.layers import Layer, GlobalAveragePooling2D, Dropout, Dense, SeparableConv2D
from tensorflow.keras.initializers import Constant
from tensorflow.keras.regularizers import l2
from tensorflow.keras.models import Model
class_names = ['ANGRY', 'DISGUST', 'FEAR', 'HAPPY', 'NEUTRAL', 'SAD', 'SURPRISE']
class ExpandDimsLayer(Layer):
    def __init__(self, axis, **kwargs):
        super().__init__(**kwargs)
        self.axis = axis

```

Cella 5 – Loss (Focal) + caricamento modello

Definisce la focal loss e carica il modello ConvNeXt da convnext_path usando custom_object_scope (necessario perché il modello usa oggetti custom).

Estratto (prime righe non vuote):

```

import numpy as np
import tensorflow as tf
def categorical_focal_loss(alpha=0.25, gamma=2.0):
    def loss(y_true, y_pred):
        y_pred = tf.clip_by_value(y_pred, tf.keras.backend.epsilon(), 1 - tf.keras.backend.epsilon())
        cross_entropy = -y_true * tf.math.log(y_pred)
        weight = alpha * tf.pow(1 - y_pred, gamma)
        focal_loss = weight * cross_entropy
        return tf.reduce_sum(focal_loss, axis=-1)
    return loss

```

Cella 6 – Data generator (bilanciato + label smoothing + paths)

Definisce un generator Keras Sequence che bilancia le classi per batch e applica label smoothing; per il test può mantenere anche i path originali delle immagini (paths_data).

Estratto (prime righe non vuote):

```

##### per abilitare i generatori di immagini
import numpy as np
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.utils import Sequence
from sklearn.utils import shuffle
class CustomBalancedDataGenerator(Sequence):
    def __init__(self, x_data, y_data, batch_size, augmentations=None, data_inf=None, label_smooth=None):
        super().__init__(**kwargs)
        self.x_data = x_data

```

```
self.y_data = y_data
```

Cella 7 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Cella 8 – Valutazione modello (predict + analisi errori)

Esegue `model.predict` sul generator test, calcola `y_pred/y_true`, identifica errori ad alta confidenza e salva figure riassuntive.

Estratto (prime righe non vuote):

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
def evaluate_keras_model(model, test_generator, model_name):
    class_names_fixed = ['ANGER', 'DISGUST', 'FEAR', 'HAPPINESS', 'NEUTRALITY', 'SADNESS', 'SURPR
    y_true = np.argmax(test_generator.y_data, axis=1)
    test_generator.in_evaluate_mode = True
    probabilities = model.predict(test_generator, verbose=1)
    test_generator.in_evaluate_mode = False
    y_pred = np.argmax(probabilities, axis=1)
```

Cella 9 – Selezione modelli (ConvNeXt-only)

Definisce la mappa dei modelli da valutare. Nel tuo caso è ridotta a ConvNeXt.

Estratto (prime righe non vuote):

```
keras_models = {
    "ConvNeXt": convnext
}
```

Cella 10 – Selezione modelli (ConvNeXt-only)

Definisce la mappa dei modelli da valutare. Nel tuo caso è ridotta a ConvNeXt.

Estratto (prime righe non vuote):

```
keras_models = {
    "ConvNeXt": convnext
}
```

Cella 11 – Export CSV risultati

Costruisce un DataFrame con (modello, immagine, true/pred, probabilità) e salva un CSV su Drive.

Estratto (prime righe non vuote):

```
import os
import numpy as np
import pandas as pd
def _decode_path(p):
    if isinstance(p, (bytes, np.bytes_)):
        return p.decode('utf-8')
    return str(p)
image_names = [os.path.basename(_decode_path(p)) for p in test_generator_focal_smoot.paths_data]
data = []
for model_name, probabilities, y_true, y_pred in model_results:
```

Cella 12 – Markdown

```
# ***PER ABILITARE IL GENERATORE***
```

Cella 13 – Markdown

```
# **PER CARICARE IL CSV**
```

Cella 14 – Export CSV risultati

Costruisce un DataFrame con (modello, immagine, true/pred, probabilità) e salva un CSV su Drive.

Estratto (prime righe non vuote):

```
import os
import numpy as np
import pandas as pd
def _decode_path(p):
    if isinstance(p, (bytes, np.bytes_)):
        return p.decode('utf-8')
    return str(p)
image_names = [os.path.basename(_decode_path(p)) for p in test_generator_focal_smoot.paths_data]
data = []
for model_name, probabilities, y_true, y_pred in model_results:
```

Cella 15 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
print(model_results)
```

Cella 16 – Markdown

```
# ***AGREEMENT ADELE***
```

Cella 17 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
import os
import pandas as pd
# Se hai le emozioni in formato TESTUALE nel CSV,
# definisci la tua lista di emozioni. Per esempio:
EMOTIONS_STR = ["ANGRY", "DISGUST", "FEAR", "HAPPY", "NEUTRAL", "SAD", "SURPRISE"]
# Se invece le etichette sono NUMERICHE (0..6) nel CSV,
# puoi definire la tua lista come:
EMOTIONS_NUM = [0, 1, 2, 3, 4, 5, 6]
def Pi_computation(csv_file, image_idx, emotions=EMOTIONS_NUM, label_column="Pred_Label"):
```

Cella 18 – Analisi downstream da CSV

Ricarica i CSV prodotti e fa analisi/plot su subset di immagini, incertezze, agreement.

Estratto (prime righe non vuote):

```
csv_file="/content/drive/MyDrive/model_results.csv" # PATH DEL CSV
df = pd.read_csv(csv_file)
```

```
unique_images = df["Image_Path"].unique()
print(len(unique_images))
```

Cella 19 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
print(df)
```

Cella 20 – Analisi downstream da CSV

Ricarica i CSV prodotti e fa analisi/plot su subset di immagini, incertezze, agreement.

Estratto (prime righe non vuote):

```
csv_file = "/content/drive/MyDrive/model_results.csv"
# Supponendo che:
# 0 -> ANGRY
# 1 -> DISGUST
# 2 -> FEAR
# 3 -> HAPPY
# 4 -> NEUTRAL
# 5 -> SAD
# 6 -> SURPRISE
# definisci la tua lista di emozioni. Per esempio:
```

Cella 21 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
print(len(results))
```

Cella 22 – Analisi agreement / IOA

Costruisce metriche/plot di agreement tra predizioni e annotazioni (e salva immagini/CSV di supporto).

Estratto (prime righe non vuote):

```
# salvare il dataframe in un file csv
df = pd.DataFrame(results)
print(df)
df.to_csv("/content/drive/MyDrive/Colab Notebooks/HPC/finale/agreement_adele/IOA_images.csv", index=False)
```

Cella 23 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
alta_concordanza = len(df[df["Pi_Overall"] > 0.9])
df_ordine = df[df["Pi_Overall"] > 0.9].sort_values(by="Pi_Overall", ascending=False)
label_map = {
    0: "ANGER",
    1: "DISGUST",
    2: "FEAR",
    3: "HAPPINESS",
    4: "NEUTRALITY",
    5: "SADNESS",
    6: "SURPRISE"
}
```

Cella 24 – Plot / visualizzazione

Celle dedicate a plot e salvataggio di immagini/figure.

Estratto (prime righe non vuote):

```
import os
import pandas as pd
import matplotlib.pyplot as plt
# Filtra il DataFrame e ordina
maggiori = df[df["Pi_Overall"] > 0] # Pi > 0
maggiori = maggiori.sort_values(by="Pi_Overall", ascending=False)
print("Numero di righe con Pi_Overall > 0:", len(maggiori))
print("Percentuale rispetto al totale:", len(maggiori)/len(df)*100, "%")
count = {}
images_names = []
```

Cella 25 – Analisi downstream da CSV

Ricarica i CSV prodotti e fa analisi/plot su subset di immagini, incertezze, agreement.

Estratto (prime righe non vuote):

```
import os
import math
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from PIL import Image
csv_file = "/content/drive/MyDrive/model_results.csv"
df_count = pd.read_csv(csv_file)
df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/HPC/finale/agreement_adele/IOA_images.csv")
# Filtra il DataFrame e ordina
```

Cella 26 – Markdown

AGREEMENT FEDE

Cella 27 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
from collections import defaultdict
error_counts = defaultdict(int)
for _, _, y_true, y_pred in model_results:
    errors = np.where(y_true != y_pred)[0] # Indici delle immagini sbagliate
    for idx in errors:
        error_counts[idx] += 1 # Conta quanti modelli sbagliano questa immagine
# ■ Stampa le immagini più difficili
sorted_errors = sorted(error_counts.items(), key=lambda x: x[1], reverse=True)
print("\n Immagini più difficili (sbagliate dal maggior numero di modelli):")
for idx, count in sorted_errors[:10]: # Mostra le prime 10
```

Cella 28 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
import numpy as np
for model_name, probabilities, y_true, y_pred in model_results:
```

```

print(f"\n Analisi delle incertezze per il modello: {model_name}\n")
for idx, (true_label, pred_label, prob) in enumerate(zip(y_true, y_pred, probabilities)):
    if pred_label != true_label: # Consideriamo solo le immagini errate
        top_2_indices = np.argsort(prob)[-2:][::-1] # Ordiniamo le due classi con probabilità
        top_2_classes = [class_names[i] for i in top_2_indices] # Convertiamo in nomi di classe
        top_2_probs = prob[top_2_indices] # Otteniamo le probabilità corrispondenti
        print(f"    Immagine {idx}:")
        print(f"    Classe reale: {class_names[true_label]}")

```

Cella 29 – Markdown

per vedere l'incertezza dei modelli tra prima e seconda classe più probabile

Cella 30 – Altro

Cella di supporto (utility, variabili, trasformazioni o analisi specifica).

Estratto (prime righe non vuote):

```
print(model_results[0])
```

Cella 31 – Plot / visualizzazione

Celle dedicate a plot e salvataggio di immagini/figure.

Estratto (prime righe non vuote):

```

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image # Per ridimensionare le immagini e creare placeholder
threshold = 0.1 # Differenza minima tra la prima e la seconda probabilità
batch_size = 18 # 6 righe x 3 colonne = 18 immagini per figura
class_names_fixed = ['ANGER', 'DISGUST', 'FEAR', 'HAPPINESS', 'NEUTRALITY', 'SADNESS', 'SURPRISE']
# Creiamo un'immagine placeholder 224x224 (bianca)
placeholder_img = Image.new('RGB', (224, 224), color=(255, 255, 255))
placeholder_array = np.array(placeholder_img)
for model_name, probabilities, y_true, y_pred in model_results:

```

Cella 32 – Markdown

per vedere se i modelli sono d'accordo tra di loro nel predire la classe errata

Cella 33 – Analisi agreement / IOA

Costruisce metriche/plot di agreement tra predizioni e annotazioni (e salva immagini/CSV di supporto).

Estratto (prime righe non vuote):

```

import numpy as np
from collections import defaultdict, Counter
import matplotlib.pyplot as plt
class_names = ['ANGRY', 'DISGUST', 'FEAR', 'HAPPY', 'NEUTRAL', 'SAD', 'SURPRISE']
# Raccogliamo i dettagli dei modelli che sbagliano
wrong_models = defaultdict(list) # key = image idx, value = list of (model_name, predicted_class)
for (model_name, probabilities, y_true, y_pred) in model_results:
    for idx, (true_label, pred_label) in enumerate(zip(y_true, y_pred)):
        if pred_label != true_label:
            wrong_models[idx].append((model_name, pred_label))

```

Note pratiche (locale vs Colab)

Il notebook è scritto in stile Colab (drive.mount, path /content/drive, !pip). Per eseguirlo in locale su VS Code devi: (1) installare TensorFlow nel tuo ambiente, (2) cambiare i path a path locali, (3) evitare drive.mount/!pip. Le celle di predict possono risultare lente o difficili da interrompere perché TensorFlow esegue molto lavoro in runtime nativo.